Reckless: Non-Functional Requirements

Measurable acceptance criteria, testing procedures and failure handling

Supplement to Section 2.4, written in response to review items 6 and 8 · November 6, 2026

1. Purpose

The review found that the non-functional requirements in Section 2.4 stated targets (60 FPS, input latency of 50 ms or less, an 80 ms input buffer, “no crashes”, “minimal learning curve”, “clear and consistent feedback”) without saying how they would be measured, which tools would be used, or what happens when a target is missed. This supplement gives each requirement a measurable acceptance criterion, a test procedure and a failure response.

Every requirement also has an implementation status taken from the current source code (Godot 4.6 project, web export). **Met** means the game already behaves this way. **Partly met** means the behaviour exists but has a known gap. **Not implemented** means the requirement is a target for future work.

2. Test environment and minimum specification

Table 2.4.0: Test environment

Item	Specification
Build under test	Web export (Godot 4.6, GL Compatibility renderer / WebGL 2), as deployed to GitHub Pages.
Logical resolution	1024 × 768, scaled to the browser window (canvas_items stretch mode).
Frame pacing	No FPS cap in the engine; frames follow the display refresh (vsync / browser requestAnimationFrame). Physics runs at 60 ticks per second.
Minimum browser	Current Chrome, Edge or Firefox with WebGL 2 and SharedArrayBuffer support. The export uses threads, which need cross-origin isolation; the bundled service worker supplies the required headers.
Minimum hardware (proposed)	Laptop or desktop with integrated graphics (Intel UHD 620 class or better), 4 GB RAM, dual-core CPU, 60 Hz display, mouse or trackpad.
Stress level	Level 10: 4 waves, 70 required kills, bursts of up to 4 objects from several spawners. All performance tests run on this level.
Test log	For every session, record the CPU, GPU, RAM, OS, browser and version, display refresh rate, and build commit hash.

3. Performance testing criteria

This table replaces the unquantified performance statements in Section 2.4 (review item 8).

Table 2.4.1: Performance Testing Criteria

Metric	Acceptance criterion	How it is measured	Tool	Status
Frame rate	Average ≥ 60 FPS; 1% low ≥ 50 FPS; never below 30 FPS	3-minute session on Level 10, FPS sampled every second (procedure 4.1)	Godot Debugger → Monitors; Chrome DevTools Performance panel; in-game FPS logger	**Partly met**
Frame time	Process + physics time ≤ 16.7 ms per frame	Same session as frame rate; record peak and average	Godot Debugger → Monitors (Process Time, Physics Process Time)	**Partly met**
Input latency	Median ≤ 50 ms from mouse click to first visible feedback; no trial over 100 ms	20 trials, engine-side and end-to-end (procedure 4.2)	Time.get_ticks_usec() timestamps; 240 fps slow-motion camera	**Partly met**
Input buffer	A slash registers if the cursor path in the last 80 ms crosses the object	Scripted fast swipes across small objects (procedure 4.3)	Manual test log	**Not implemented**
Hit registration	≥ 95% of deliberate slashes and smashes register	100 attempts at slow, medium and fast swipe speeds (procedure 4.3)	Manual test log	**Partly met**
Load time	Level scene ready ≤ 3 s after selection on minimum hardware	Stopwatch from button press to countdown start, 5 trials	Stopwatch / DevTools	**Partly met**

***Partly met** in this table means the game is built to meet the target, but no measurement has been recorded yet. A row becomes **Met** once its procedure passes on the minimum-specification machine.*

4. Testing procedures

4.1 Frame rate and frame time

1. Close other browser tabs and applications. Note the machine details in the test log.
2. Open the deployed web build. Start Level 10 with no powerup.
3. Play for 3 minutes, or until the level ends, while the FPS logger writes one sample per second (*Engine.get_frames_per_second()*) and the per-frame process time.
4. Repeat 3 times. Report average FPS, 1% low FPS, minimum FPS and peak frame time.
5. Pass if every run meets the criteria in Table 2.4.1.

When testing from the Godot editor, the Debugger → Monitors tab shows FPS, Process Time and Physics Process Time live. In the browser, a Chrome DevTools Performance recording shows dropped and long frames.

4.2 Input latency

Latency is measured two ways, because the engine can only see part of the delay.

- **Engine-side latency.** A debug script records *Time.get_ticks_usec()* when the click event arrives, and again on the next *RenderingServer.frame_post_draw* signal after the hit feedback is created. The difference is the time the game itself adds. Target: one frame (≤ 17 ms) or less.

- **End-to-end latency.** Film the mouse and screen together with a phone camera in 240 fps slow motion (one video frame = 4.17 ms). Count frames from the moment the mouse button goes down to the first frame showing particles or the split object. Run 20 trials and report the median and the worst case. This includes the operating system, browser and display delay.

4.3 Input buffer and hit registration

1. Use the test level (scenes/levels/test_level) with a single spawner throwing blue objects only.
2. Perform 100 slashes: about one third slow, one third medium and one third as fast as possible.
3. Perform 100 single-click smashes on red objects.
4. Record each attempt as hit or miss. Pass if at least 95 of each 100 register.

Current behaviour: *a slash only registers when the cursor is outside the object on one frame and inside it on the next. A very fast swipe can move past a small object between two frames and miss it. The 80 ms input buffer is the planned fix: keep the cursor positions from the last 80 ms and test the whole path against the object, not just the current point.*

5. Failure handling on minimum-specification hardware

If a performance target is missed during testing, the team responds as follows.

Table 2.4.2: Failure handling

Condition	Response	Status
Average FPS below 60 but at least 50	Accept for release. Record it in the test log as a known limitation for that hardware.	**Met**
Average FPS below 50, or 1% low below 30	Add a “Reduced effects” option in Settings that lowers particle counts and turns off screen shake, background parallax and the mouse trail. Re-test with the option on.	**Not implemented**
Still below 30 FPS with reduced effects	Lower the spawners’ max-alive caps on the affected levels, or raise the minimum specification and state it on the game page.	**Planned**
Input latency median over 50 ms	Check that feedback is created in the same frame as the hit (Section 8). If the engine-side delay is within one frame, record the remainder as browser or display delay.	**Partly met**
Hit registration below 95%	Implement the 80 ms input buffer (Section 4.3) and repeat the test.	**Not implemented**
Browser lacks WebGL 2 or SharedArrayBuffer	The game cannot start. The page should show a message naming a supported browser.	**Partly met**

6. Reliability requirements

Current statement: “The game should operate consistently without crashes or data loss.” The game has no crash reporting or analytics, so a field metric such as “crash rate per 1000 sessions” cannot be measured. The criteria below can be checked by testing.

Table 2.4.3: Reliability requirements

Requirement	Acceptance criterion	Verification	Status
No crashes	0 crashes across the full test matrix: tutorial, Levels 1–10, shop, level menu, settings, pause/resume, game over and retry. A crash is any error that closes or freezes the game or needs a page reload.	One full pass of the matrix per release, recorded in the test log	**Partly met**
Stability over time	30 minutes of continuous play with no crash and no steady growth in memory or object count	Godot Monitors: Static Memory and Object Count, sampled every minute	**Partly met**
Save on every change	Progress is written to disk immediately after each gem pickup, level unlock, shop purchase, powerup use and settings change	Code inspection; close the tab right after each action, reopen and check the value	**Met**
Save round-trip	100% of saved fields read back with the same values after a reload	20 save/reload cycles covering every field	**Met**
Corrupt or old save file	The game starts with default values instead of crashing; renamed items and currency carry over	Replace the save file with invalid data, and with a save from an older build, then launch	**Met**
Save failure	If the save file cannot be opened (for example, storage blocked in a private window), the game keeps running and tells the player progress will not be kept	Launch in a private window with site storage blocked	**Not implemented**

Notes on data persistence

- A 30-second auto-save is not needed: the game already saves after every state change, which loses less progress than a timer would.
- The score and progress inside a level are not saved. Leaving mid-level loses that run, but gems already earned are kept.
- In the web build, saves live in the browser's IndexedDB storage. Clearing site data for the game's address erases progress.
- Two separate files are written: game progress (gems, unlocked levels, inventory, tutorial flags) and settings (volume levels).

7. Learning curve

Current statement: "Minimal learning curve." Level 1 opens with a step-by-step tutorial that points out the pause button, lives, wave label and progress bar, then introduces the three object colours. A separate tutorial level is also available. Level 1 has three waves needing 5, 10 and 10 kills (25 in total), spawning 1–2 objects every 2–3 seconds; the first wave is blue only, the second red only, and green appears in the third. From this configuration, a first clear is estimated at 1 to 1.5 minutes, including the tutorial prompts. This estimate has not yet been confirmed by playtesting.

Table 2.4.4: Usability requirements

Requirement	Acceptance criterion	Verification
Time to first clear	At least 80% of first-time players complete Level 1 within 2 minutes of starting it	Playtest with 5 or more people who have not played the game; time from level start to the win screen
Attempts	At least 80% of first-time players clear Level 1 in 2 attempts or fewer	Same playtest; count retries
Rules understood	After Level 1, at least 80% can say which colour to slash, which to smash and which to avoid	Three-question check after the playtest
No outside help	No tester asks how to play during Level 1	Observer notes

If a criterion fails, the team revises the tutorial text or slows the Level 1 spawn rate, then repeats the playtest with new testers.

8. Feedback response time standards

Current statement: “Clear and consistent feedback.” **Standard:** all gameplay feedback starts within one frame (≤ 17 ms at 60 FPS) of the action that triggers it, and each effect lasts the time listed below. These values are set in the game’s code and are verified by frame-stepping a screen recording.

Table 2.4.5: Feedback response times

Event	Feedback	Start	Duration
Slash (blue) or smash (red)	Particles, the object splits into pieces, slash/smash sound	Same frame	Particle lifetime
Impact sound	Second sound layer on every slash or smash	+30 ms	Length of sound
Mistake or missed object	Red screen flash and camera shake; a heart (or shield) is removed	Same frame	Flash fades over 0.3 s; shake 0.25 s
Object thrown	Throw sound and a coloured glow at the spawn point	Same frame	Glow fades over 0.6 s
Wave complete	“Wave N Completed!” banner	Same frame	3 blinks over 3 s; next wave after 2.5 s
Level start / resume	Countdown with a beep each second	Immediately	3 s
Win or game over	Victory or defeat sound and end screen	After 1.2 s	Until the player chooses

9. Review checklist

Review item	Addressed in
How the performance targets will be measured	Table 2.4.1; Section 4
Tools used for testing	Table 2.4.1; Section 4
What happens if targets are not met on minimum hardware	Section 2 (minimum specification); Table 2.4.2
Measurable acceptance criteria for all non-functional requirements	Tables 2.4.1, 2.4.3, 2.4.4, 2.4.5
Testing procedures	Section 4; verification columns in Tables 2.4.3 and 2.4.4
Failure handling	Table 2.4.2; save failure row in Table 2.4.3; Section 7 note
Crash rate, learning curve and feedback quantified (item 6)	Sections 6, 7 and 8

Open implementation work

- 80 ms input buffer for slashes (Section 4.3).
- Check for a failed save and warn the player (Table 2.4.3).
- “Reduced effects” option in Settings (Table 2.4.2).
- FPS and latency debug logger used by procedures 4.1 and 4.2.